

Análise Léxica

Disciplina: Compiladores e Interpretadores
Projeto integrador: linguagem MINIC

na a implementação de um analisador léxico, pois ao invés de reescrever os padrões antigos, e não o programa todo. Essa abordagem também acelera o processo de implementação do analisador léxico, pois o programador especifica os padrões do software em bem alto nível e conta com o gerador para produzir o código detalhado. Na Seção 3.5, apresentamos um gerador de analisador léxico chamado *Lex* (ou *Flex*, em uma versão mais recente).

Começamos o estudo dos geradores de analisador léxico apresentando as expressões regulares, uma notação conveniente para especificar os padrões dos lexemas. Mostramos como essa notação pode ser transformada, primeiro em autômatos não deterministas e depois em autômatos deterministas. Essas duas notações podem ser usadas como entrada para um “driver”, ou seja, um código que simula esses autômatos e os utiliza como guia para determinar o próximo token. Esse driver e a especificação na forma de autômato formam o núcleo do analisador léxico.

3.1 O PAPEL DO ANALISADOR LÉXICO

Como a primeira fase de um compilador, a tarefa principal do analisador léxico é ler os caracteres da entrada do programa fonte, agrupá-los em lexemas e produzir como saída uma sequência de tokens para cada lexema no programa fonte. O fluxo de tokens é enviado ao analisador sintático para que a análise seja efetuada. É comum que o analisador léxico interaja com a tabela de símbolos também. Por exemplo, quando o analisador léxico descobre que um lexema é um identificador, precisa inserir esse lexema na tabela de símbolos. Em alguns casos, as informações referentes ao identificador devem ser lidas da tabela de símbolos pelo analisador léxico para ajudá-lo a determinar o token apropriado que ele precisa passar ao analisador sintático.

Essas interações são mostradas na Figura 3.1. Normalmente, a interação é implementada fazendo-se com que o analisador sintático chame o analisador léxico. A chamada, sugerida pelo comando *getNextToken*, faz com que o analisador léxico leia caracteres de sua entrada até que ele possa identificar o próximo lexema e produza para ele o próximo token, que retorna ao analisador sintático.

token

Figura de abertura: interação entre analisador léxico, analisador sintático e tabela de símbolos. Fonte: Aho et al., Cap. 3, Fig. 3.1, p. 70.

Objetivo da aula: compreender como o fluxo de caracteres do programa-fonte é agrupado em lexemas e convertido em tokens para a análise sintática.

1. Objetivos de aprendizagem

Ao final desta aula, você deverá ser capaz de:

- explicar o papel do analisador léxico no compilador;
- diferenciar token, padrão, lexema e atributo;
- classificar tokens da linguagem MINIC;
- descrever o tratamento de espaços, comentários e posições no código-fonte;
- propor mensagens para erros léxicos;
- relacionar o scanner à tabela de símbolos e ao parser.

2. O papel do analisador léxico

O analisador léxico é a primeira fase do compilador. Ele lê os caracteres do programa-fonte, agrupa-os em sequências significativas — os lexemas — e produz uma sequência de tokens. Essa sequência é entregue ao analisador sintático, que trabalhará com símbolos abstratos em vez de caracteres individuais.

Na implementação de um analisador léxico, pois só temos de reconhecer os padrões aceitos, e não o programa todo. Esta abordagem também acelera o processo de implementação do analisador léxico, pois o programador especifica os padrões do software em bem alto nível e conta com o gerador para produzir o código detalhado. Na Seção 3.5, apresentamos um gerador de analisador léxico chamado *Lex* (ou *Flex*, em uma versão mais recente).

Começamos o estudo dos geradores de analisador léxico apresentando as expressões regulares, uma notação conveniente para especificar os padrões dos lexemas. Mostramos como essa notação pode ser transformada, primeiro em autômatos não deterministas e depois em autômatos deterministas. Essas duas notações podem ser usadas como entrada para um “driver”, ou seja, um código que simula esses autômatos e os utiliza como guia para determinar o próximo token. Esse driver e a especificação na forma de autômato formam o núcleo do analisador léxico.

3.1 O PAPEL DO ANALISADOR LÉXICO

Como a primeira fase de um compilador, a tarefa principal do analisador léxico é ler os caracteres da entrada do programa fonte, agrupá-los em lexemas e produzir como saída uma sequência de tokens para cada lexema no programa fonte. O fluxo de tokens é enviado ao analisador sintático para que a análise seja efetuada. É comum que o analisador léxico interaja com a tabela de símbolos também. Por exemplo, quando o analisador léxico descobre que um lexema é um identificador, precisa inserir esse lexema na tabela de símbolos. Em alguns casos, as informações referentes ao identificador devem ser lidas da tabela de símbolos pelo analisador léxico para ajudá-lo a determinar o token apropriado que ele precisa passar ao analisador sintático.

Essas interações são mostradas na Figura 3.1. Normalmente, a interação é implementada fazendo-se com que o analisador sintático chame o analisador léxico. A chamada, sugerida pelo comando *getNextToken*, faz com que o analisador léxico leia caracteres de sua entrada até que ele possa identificar o próximo lexema e produza para ele o próximo token, que retorna ao analisador sintático.

token

Fig. 1 — Fluxo de tokens e interação com a tabela de símbolos. Fonte: Aho et al., Fig. 3.1, p. 70.

Regra prática: o scanner responde “que unidade léxica é esta?”; o parser responde “como essas unidades se organizam?”.

3. Token, padrão, lexema e atributo

Esses quatro conceitos devem ser separados:

Conceito	Definição	Exemplo MINIC
Token	Nome abstrato, com atributo opcional, usado pelo parser.	IDENTIFIER, INTEGER, PLUS
Padrão	Descrição da forma que os lexemas de um token podem assumir.	[A-Za-z_][A-Za-z0-9_]*
Lexema	Sequência concreta de caracteres que casa com um padrão.	total, contador2, _x
Atributo	Informação adicional associada ao token.	valor 42 ou entrada na tabela de símbolos

Ao discutir a análise léxica, usamos três termos relacionados, porém distintos:

- Um *token* é um par consistindo em um nome e um valor de atributo opcional. O nome do token é um símbolo abstrato que representa um tipo de unidade léxica, por exemplo, uma palavra-chave em particular, ou uma sequência de caracteres da entrada denotando um identificador. Os nomes de token são os símbolos da entrada que o analisador sintático processa. No texto a seguir, geralmente escrevemos o nome de um token em negrito. Vamos freqüentemente nos referir a um token por seu nome de token.
- Um *padrão* é uma descrição da forma que os lexemas de um token podem assumir. No caso de uma palavra-chave como um token, o padrão é apenas uma sequência de caracteres que formam uma palavra-chave. Para identificadores e alguns outros tokens, o padrão é uma estrutura mais complexa, que é *casada* por muitas sequências de caracteres.
- Um *lexema* é uma sequência de caracteres no programa fonte que casa com o padrão para um token e é identificado pelo analisador léxico como uma instância desse token.

EXEMPLO 3.1: A Figura 3.2 mostra alguns tokens típicos, seus padrões descritos informalmente, e alguns exemplos de lexemas. Para ver como esses conceitos são usados na prática, no comando em C

```
printf("Total = %d\n", score);
```

tanto `printf` quanto `score` são lexemas casando com o padrão para o token **id**, e `"Total = %d\n"` é um lexema casando com **literal**.

TOKEN	DESCRIÇÃO INFORMAL	EXEMPLOS DE LEXEMAS
-------	--------------------	---------------------

Fig. 2 — Exemplos de tokens, padrões e lexemas. Fonte: Aho et al., Fig. 3.2, p. 71.

Para o projeto MINIC, um identificador pode ser representado como <IDENTIFIER, entrada>; um número pode ser <INTEGER, 42>. Operadores e delimitadores normalmente não precisam de atributo.

4. Classes de tokens do MINIC

Classe	Exemplos	Expressão regular ou regra
Palavras reservadas	int, if, while, return	sequência literal da palavra
Identificadores	soma, _valor, contador2	[A-Za-z_][A-Za-z0-9_]*
Inteiros	0, 42, 1000	[0-9]+
Reais	3.14, 0.5	[0-9]+\.[0-9]+
Operadores	+, -, *, /, ==, <=	regras específicas; testar os mais longos primeiro
Delimitadores	(){}[]; ,	um token por símbolo
Comentários	// ... e /* ... */	descartar antes de emitir tokens
Cadeias/caracteres	"texto", 'A'	delimitação e escapes válidos

A ordem das regras importa. Por exemplo, == deve ser reconhecido antes de =; caso contrário, o scanner pode produzir dois tokens de atribuição em vez de um operador de igualdade.

5. Exemplo de tokenização

```
position = initial + rate * 60;
```

Sequência de tokens MINIC:

```
<IDENTIFIER, position> ASSIGN <IDENTIFIER, initial> PLUS <IDENTIFIER, rate> STAR  
<INTEGER, 60> SEMICOLON
```

Os espaços são descartados, mas a linha e a coluna de cada token devem ser preservadas para os diagnósticos.

6. O analisador léxico e a tabela de símbolos

Quando encontra um identificador, o scanner pode inserir seu lexema na tabela de símbolos ou recuperar uma entrada já existente. O token transporta, então, uma referência à entrada. Informações como nome, tipo, escopo e posição da declaração serão usadas posteriormente pela análise semântica e pela geração de código.

Uma entrada MINIC pode conter:

- nome do identificador;
- categoria: variável, função, parâmetro ou vetor;
- tipo e dimensões, quando aplicável;
- escopo e profundidade lexical;
- linha e coluna da declaração;
- informações de armazenamento, quando o back-end estiver disponível.

7. Espaços, comentários e posição

Espaços, tabulações e quebras de linha normalmente funcionam como separadores e não geram tokens.

Comentários de linha começam com `//`; comentários de bloco começam com `/*` e terminam com `*/`. O scanner deve atualizar linha e coluna mesmo ao descartar esses elementos.

```
int x = 10; // comentário descartado /* bloco descartado */ x = x + 1;
```

8. Erros léxicos

Um erro léxico ocorre quando nenhum padrão reconhece o próximo prefixo da entrada. Exemplos: símbolo não permitido, cadeia não terminada, literal malformado ou comentário de bloco sem fechamento. A estratégia de recuperação mais simples é o modo de pânico: descartar caracteres até encontrar um token válido.

um analisador léxico não tem como saber se `fi` é a palavra-chave `if` escrita errada ou um identificador de função não declarada. Como `fi` é um lexema válido para o token `id`, o analisador léxico precisa retornar o token `id` ao analisador sintático e deixar que alguma outra fase do compilador — provavelmente o analisador sintático, neste caso — trate do erro devido à transposição das letras.

Suponha, porém, que ocorra uma situação na qual o analisador léxico seja incapaz de prosseguir porque nenhum dos padrões para tokens casa com nenhum prefixo da entrada restante. A estratégia de recuperação mais simples é a recuperação no “modo de pânico”. Removemos os caracteres seguintes da entrada restante, até que o analisador léxico possa encontrar um token bem formado no início da entrada que resta. Essa técnica de recuperação pode confundir o analisador sintático, mas, em um ambiente de computação interativo, pode ser bastante adequada.

Outras ações de recuperação de erro possíveis são:

1. Remover um caractere da entrada restante.
2. Inserir um caractere que falta na entrada restante.
3. Substituir um caractere por outro caractere.
4. Transpor dois caracteres adjacentes.

Fig. 3 — Discussão de dificuldades e recuperação de erros léxicos. Fonte: Aho et al., Cap. 3, pp. 72–73.

9. Leitura antecipada e buffering

Para decidir se um lexema terminou, o scanner frequentemente precisa olhar um ou mais caracteres à frente. Isso acontece em identificadores, números e operadores que possuem versões de um e dois caracteres, como = e ==. Técnicas de buffering reduzem o custo de ler repetidamente a entrada.

FIGURA 3.3 Usando um par de buffers de entrada.

Cada buffer possui o mesmo tamanho N , e N normalmente corresponde ao tamanho de um bloco de disco, por exemplo, 4096 bytes. Usando um comando de leitura do sistema, podemos ler N caracteres para um buffer, em vez de fazer uma chamada do sistema para cada caractere. Se restarem menos de N caracteres no arquivo de entrada, então um caractere especial, representado por `eof`, marca o fim do arquivo fonte e é diferente de qualquer caractere possível do programa fonte.

São mantidos dois apontadores para a entrada:

1. O apontador `lexemeBegin` marca o início do lexema corrente, cuja extensão estamos tentando determinar.
2. O apontador `forward` lê adiante, até que haja um casamento de padrão; a estratégia exata de como isso é determinado será explicada no restante deste capítulo.

Uma vez que o próximo lexema é determinado, `forward` é configurado para apontar para o seu último caractere à direita. Em seguida, após o lexema ser registrado como um valor de atributo de um token retornado ao analisador sintático, `lexemeBegin` é configurado para apontar para o caractere imediatamente após o lexema recém-encontrado. Na Figura 3.3, vemos que o apontador `forward` passou do fim do próximo lexema, `**` (o operador de exponenciação em Fortran), e precisa ser recuado uma posição à sua esquerda.

Avançar o apontador `forward` exige que primeiro testemos se chegamos ao fim de um dos buffers e, nesse caso, precisamos recarregar o outro buffer da entrada e mover o apontador `forward` para o início do buffer recém-carregado. Se nunca precisarmos examinar tão adiante do lexema corrente que a soma do tamanho do lexema com a distância que examinamos adiante é maior do que N , nunca sobrescreveremos o lexema no buffer antes de determiná-lo.

Podemos esgotar o espaço em buffer?

Na maioria das linguagens modernas, os lexemas são pequenos, e um ou dois caracteres de lookahead são suficientes. Assim, um buffer de tamanho N na casa dos milhares é suficiente, e o esquema de buffer duplo da Seção 3.2.1 funciona sem problemas. Existem, porém, alguns riscos. Por exemplo, se as cadeias de caracteres puderem ser muito grandes, estendendo-se por muitas linhas, talvez nos depararemos com a possibilidade de um lexema ser maior do que N . Para evitar problemas com cadeias de caracteres longas, podemos tratá-las como uma concatenação de componentes, um de cada linha na qual a cadeia de caracteres é escrita. Por exemplo, Java adota a convenção de representar cadeias de caracteres longas

Fig. 4 — Exemplo de buffering de entrada para análise léxica. Fonte: Aho et al., Fig. 3.3, p. 74.

Na implementação didática do MINIC, é suficiente começar com uma leitura sequencial e um ponteiro de lookahead. A técnica de dois buffers e sentinelas pode ser estudada como otimização posterior.

10. Atividade prática — scanner do MINIC

Implemente ou especifique um scanner para o trecho:

```
int main() { int total = 2 + 3 * 4; // mostra o resultado print(total); return 0; }
```

Registre, para cada token: nome, lexema, linha, coluna e atributo quando necessário. Depois, introduza deliberadamente um símbolo inválido e uma cadeia não terminada. Observe como o diagnóstico é produzido.

Checklist da Etapa 1

- palavras reservadas não são confundidas com identificadores;
- operadores de dois caracteres são reconhecidos corretamente;
- comentários e espaços são descartados;
- linha e coluna são preservadas;
- erros produzem mensagens úteis;
- há testes positivos e negativos.

11. Exercícios de fixação

1. Identifique tokens, lexemas e atributos na expressão `resultado = a + b * 10;`.
2. Escreva expressões regulares para identificadores, inteiros e reais do MINIC.
3. Explique por que `==` deve ser testado antes de `=`.
4. Quais elementos abaixo devem ser descartados pelo scanner? Espaço, comentário, identificador, quebra de linha, operador, cadeia de caracteres.
5. Proponha uma mensagem para o erro em `int x = 10 @ 2;`. Inclua linha, coluna, lexema e sugestão.
6. Para `x = y + 3;`, indique quais informações podem ser armazenadas na tabela de símbolos para `x` e `y`.

Síntese

O analisador léxico transforma caracteres em uma sequência estruturada de tokens. Sua qualidade influencia a simplicidade do parser, a precisão dos diagnósticos e a eficiência do compilador. No MINIC, o scanner será a primeira entrega independente e a base para as etapas seguintes.

Leitura recomendada

Aho, Lam, Sethi e Ullman. *Compiladores: Princípios, Técnicas e Ferramentas*, 2ª edição. Capítulo 3 — Análise léxica: Seção 3.1, “O papel do analisador léxico”, pp. 70–73; Seção 3.2, “Bufferes de entrada”, pp. 73–75. Para continuidade: Seção 3.3, “Especificação de tokens”, pp. 75–81.

Nota sobre as imagens

As figuras foram reproduzidas em escala reduzida exclusivamente para fins didáticos, com indicação da obra e das páginas de origem. O texto desta apostila é material de apoio original e não substitui a leitura do livro.